Explainable Counterfeit Medicine Detection with Blockchain Audit Trail

Complete Project Document: PRD + TRD + 12-Phase Implementation Plan

Item	Value
Project type	College project (team of 2)
Approach	Option C: CNN + Explainable AI + autonomous decision logic + Ethereum smart-contract audit trail + Streamlit app
Duration	~28 days
Compute	Laptop, CPU-friendly (GPU optional), everything run locally via Claude Code in the terminal
Blockchain	Local Hardhat node (chain ID 31337), no real money involved

Table of Contents

- **Part A: Product Requirements Document (PRD)**
- **Part B: Technical Requirements Document (TRD)**
- **Part C: The 12-Phase Implementation Plan** (what happens, how, files, prompts, exit criteria)
- **Part D: Timeline, Risks, Definition of Done**
- **Appendix: Viva Q&A, Glossary**

PART A: PRODUCT REQUIREMENTS DOCUMENT (PRD)

A1. Overview and Problem Statement

Counterfeit and substandard medicines are a serious public-health and economic problem. Fake tablets and packaging often look nearly identical to genuine ones, and inspectors at pharmacies, warehouses, and customs cannot send every item to a laboratory. Existing AI-based detectors tend to have two weaknesses:

1. **Black-box decisions.** A model says "fake" with no reason, so an inspector cannot trust or challenge it.
2. **No trustworthy record.** Decisions are stored in ordinary databases or files that can be silently edited afterwards, so there is no reliable audit trail.

This project builds a prototype that fixes both: it detects counterfeit medicine images, shows why it decided (heatmap explanations), decides autonomously when confident and escalates to a human when not, and writes a tamper-evident record of every decision to a blockchain smart contract.

A2. Vision, Goals, Non-Goals

Vision

A transparent, accountable, semi-autonomous inspection assistant: **every decision is explainable and every record is verifiable.**

Goals

ID	Goal
G1	Classify a medicine/pill image as genuine or counterfeit with high recall on counterfeit
G2	Provide a visual explanation (Grad-CAM, compared against SHAP/LIME) for each decision
G3	Take autonomous action (APPROVE / FLAG / ESCALATE) from model confidence
G4	Record each decision immutably on a smart contract (hashes only) with access control
G5	Allow anyone to verify an image against its on-chain record
G6	Deliver a working, demonstrable web app and a complete academic report

Non-Goals (explicitly out of scope)

- Detecting real-world counterfeits (no labelled public dataset exists; counterfeits are **simulated**)
- Chemical or spectroscopic analysis
- Deployment on a public chain or mainnet
- Storing images on-chain or on IPFS
- Mobile app, multi-user login system, supply-chain batch tracking
- Regulatory or clinical validation

A3. Users and Personas

Persona	Description	Needs
Inspector / Pharmacist (primary)	Checks incoming medicine stock	Quick verdict, understandable reason, clear next action, proof the check was recorded
Auditor / Regulator	Reviews past decisions	Browse the ledger, verify that a record matches an image, trust that records were not edited
System Admin	Owns the contract	Authorise or revoke accounts that may log decisions
Evaluator / Examiner (viva)	Assesses the project	Clear demo, evidence of metrics, honest limitations

A4. User Stories

ID	Story	Acceptance
US1	As an inspector, I upload a medicine image and see a verdict with confidence	Verdict, probability, and action shown within a few seconds
US2	As an inspector, I see a heatmap showing what influenced the decision	Heatmap overlay shown next to original
US3	As an inspector, uncertain cases are sent to me instead of auto-decided	Items between thresholds show ESCALATE
US4	As an auditor, I view all recorded decisions	Ledger table lists all on-chain records
US5	As an auditor, I upload an image and a record ID to check integrity	App returns MATCH or MISMATCH
US6	As an admin, I control who may log decisions	Unauthorised account's transaction is rejected
US7	As an inspector, I get a clear message if the blockchain is unavailable	No crash; explanatory message shown
US8	As an examiner, I can see model metrics and limitations	About/Metrics tab displays them, including the simulated-data disclaimer

A5. Functional Requirements

Priority: **M** = Must, **S** = Should, **C** = Could.

ID	Requirement	Priority
FR-01	System shall accept JPG/PNG image upload	M
FR-02	System shall output P(counterfeit) in [0,1] using a CNN	M
FR-03	System shall generate a Grad-CAM heatmap overlay for each prediction	M
FR-04	System shall map probability to an action using two configurable thresholds	M
FR-05	System shall compute SHA-256 of the original image bytes and the heatmap file bytes	M
FR-06	System shall submit image hash, heatmap hash, verdict, confidence to the smart contract	M
FR-07	Smart contract shall store records append-only with timestamp and submitter	M
FR-08	Smart contract shall restrict logging to authorised accounts	M
FR-09	System shall return transaction hash and record ID after logging	M
FR-10	System shall verify an uploaded image against an on-chain record by hash comparison	M
FR-11	App shall list all on-chain records	M
FR-12	System shall handle chain-unavailable, stale-contract, and invalid-image errors gracefully	M
FR-13	System shall provide SHAP or LIME explanations for sample images (offline analysis)	S
FR-14	System shall support a fallback hash-chain audit backend selectable by config	S
FR-15	System shall record gas used and processing time per decision	S
FR-16	App shall allow CSV download of the ledger	C
FR-17	App shall show an About tab with metrics and limitations	S
FR-18	Admin shall be able to revoke an authorised account	C

A6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	End-to-end decision (inference + heatmap + hashing + chain tx) ⩽ ~5 s on a laptop CPU
NFR-02	Accuracy	Counterfeit recall ⩾ 0.90 on the simulated test set (target; report honestly if lower)
NFR-03	Explainability	Mean Grad-CAM overlap with altered region reported; qualitative figures for ⩾ 6 cases
NFR-04	Integrity	Any change to the image after logging must cause verification to fail
NFR-05	Reproducibility	Fixed random seeds; `requirements.txt`; documented run order
NFR-06	Usability	Colour-coded actions; plain-language explanation; one-screen result
NFR-07	Portability	Runs on Windows/Mac/Linux with Python 3.10+ and Node LTS
NFR-08	Privacy	Only hashes stored on-chain; no image data leaves the machine
NFR-09	Maintainability	Modular code; audit backend swappable via two functions
NFR-10	Cost	No paid services; test ETH only

A7. User Flows

Flow 1: Check a medicine

```
Open app → Tab "Check medicine" → Upload image
  → App shows: verdict, confidence, original | heatmap, action badge
  → App shows: transaction hash + record ID  (or error message if chain is down)
```

Flow 2: Audit the ledger

```
Tab "Blockchain ledger" → Refresh → Table of records (id, time, verdict, confidence,
hashes, submitter)
```

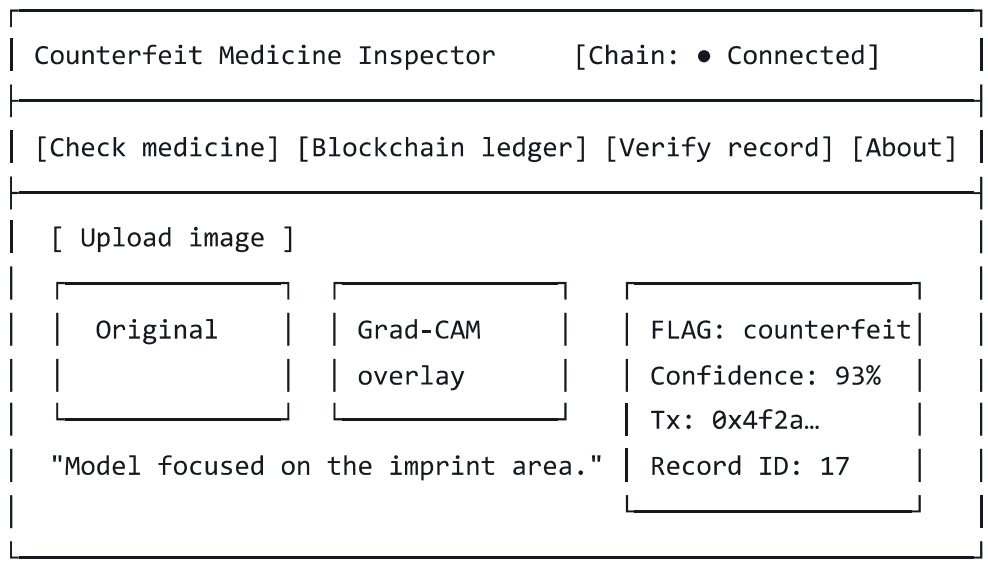
Flow 3: Verify a record

```
Tab "Verify record" → Upload image + enter record ID
  → App re-hashes image, reads on-chain record, compares
  → MATCH (green) or MISMATCH (red)
```

Flow 4: Admin authorisation (script-level)

```
Owner account calls authorize(address) → address can now log decisions
Unauthorised account calls logDecision → transaction reverts "Not authorized"
```

A8. UI Requirements (low-fidelity wireframe)



Colours: APPROVE = green, FLAG = red, ESCALATE = amber. Sidebar shows chain status, contract address, active account, and thresholds.

A9. Success Metrics and Acceptance Criteria

Metric	Target
Counterfeit recall (test)	⩾ 0.90
Overall accuracy / F1 (test)	Reported with confusion matrix
Tamper test	100% of modified images produce MISMATCH
Access control test	Unauthorised log transaction reverts
Automation rate	% of test cases decided without escalation, reported with error rate
Wrong auto-decisions	Minimise missed fakes among APPROVE decisions; reported
Latency	⩽ ~5 s per decision
Gas per record	Measured and reported (expect order of 10^5 gas)
Demo	Full viva script runs without errors

A10. Assumptions, Constraints, Dependencies

- A suitable public pill/packaging image dataset is available on Kaggle (500–1000 images).
- Counterfeits are synthetic alterations of genuine images; findings do not claim real-world performance.
- Kaggle download is done manually in a browser.
- Node.js LTS, Python 3.10+, and internet access for package installation exist.
- The Hardhat node resets on restart; the contract is redeployed each session.
- Hardhat test keys are public knowledge and must never be used with real funds.

A11. Product Risks

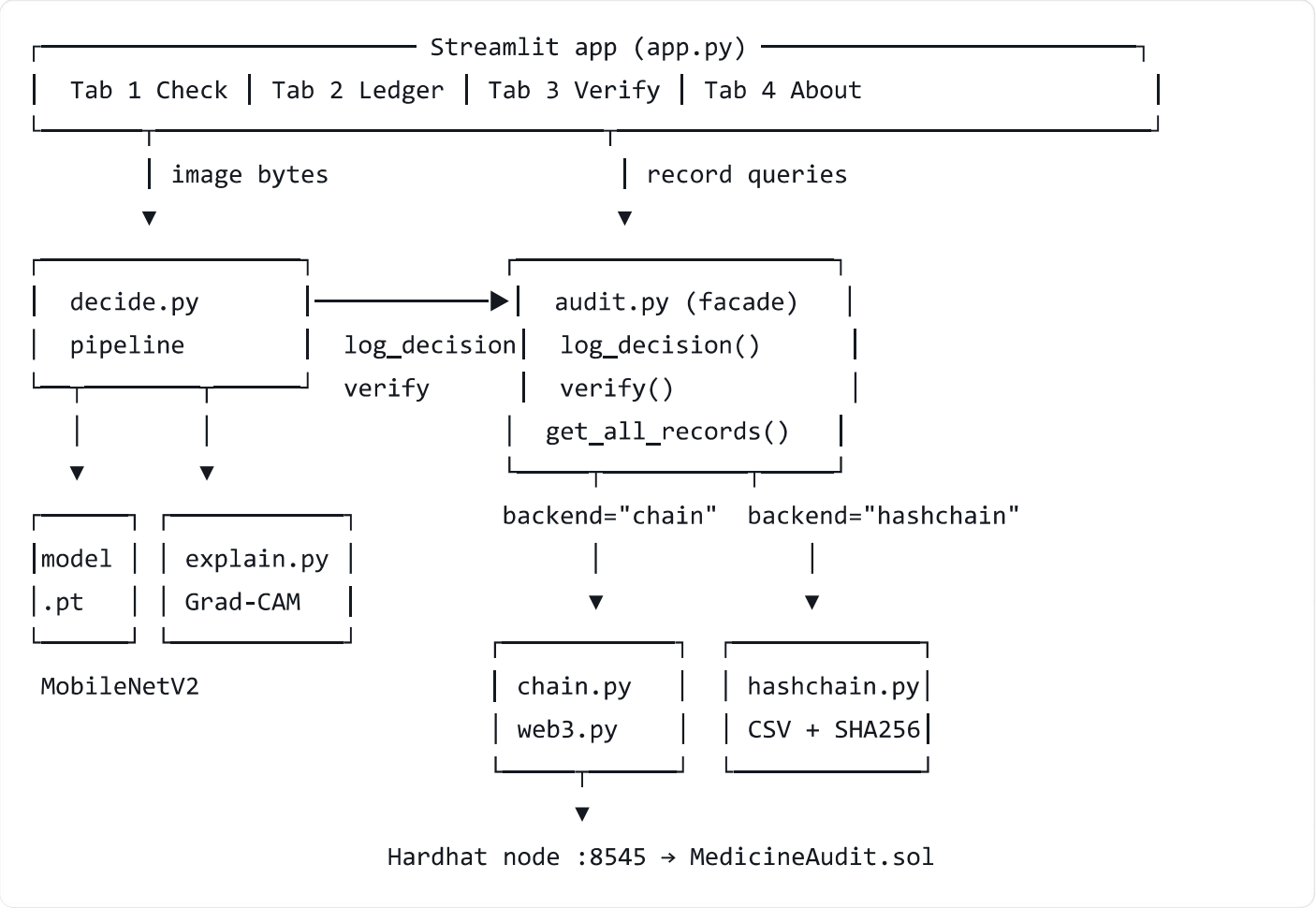
See Part D (risk table). Top three: blockchain tooling delay, fakes too easy (inflated metrics), shortcut learning.

A12. Release Scope

MVP (must be demo-ready by Day 21): upload → verdict → heatmap → action → on-chain record → ledger → verify. **Full (Day 28):** metrics, SHAP comparison, gas/timing stats, robustness tests, report, logbook, viva readiness.

PART B: TECHNICAL REQUIREMENTS DOCUMENT (TRD)

B1. Architecture Overview



Layers

Layer	Components	Responsibility
Data	`generate_fakes.py`, `dataset.py`	Build genuine/counterfeit splits, load tensors
ML	`train.py`, `model.pt`	Train and evaluate CNN
Explainability	`explain.py`	Grad-CAM overlays, SHAP/LIME, overlap scoring
Decision	`decide.py`	Thresholds → action; orchestration
Audit	`audit.py`, `chain.py`, `hashchain.py`, contract	Record and verify decisions
Presentation	`app.py`	User interface

B2. End-to-End Walkthrough (what happens at each step)

This traces **one image** through the whole system, from upload to on-chain proof.

#	Step	What happens	Where
1	Upload	User selects JPG/PNG. Streamlit gives raw bytes (`uploaded.getvalue()`). These **original bytes are kept untouched** for hashing.	`app.py`
2	Decode and validate	Bytes opened with PIL, converted to RGB. Invalid or corrupt files raise a clear error.	`decide.py`
3	Preprocess	Resize to 160×160, convert to tensor, normalise with ImageNet mean/std.	`dataset.py`
4	Inference	MobileNetV2 outputs 2 logits. Softmax gives class probabilities. The counterfeit probability is read from the correct class index.	`train.py` / `decide.py`
5	Explanation	Grad-CAM uses the last conv block's activations and gradients for the predicted class to build a heatmap, overlaid on the image, then encoded as PNG bytes.	`explain.py`
6	Decision	$p \geq 0.85 \rightarrow$ FLAG (verdict 1); $p \leq 0.15 \rightarrow$ APPROVE (verdict 0); otherwise ESCALATE (verdict 2).	`decide.py`
7	Hashing	`SHA-256(original image bytes)` and `SHA-256(heatmap PNG bytes)` produce two 32-byte digests.	`chain.py`
8	On-chain write	web3.py calls `logDecision(imageHash, heatmapHash, verdict, confidence)`. The node mines a block and the contract appends a `Record` with timestamp and sender.	`chain.py`, contract

#	Step	What happens	Where
9	Receipt	Python waits for the receipt and reads the transaction hash, gas used, block number, and the new record ID (from the event or `totalRecords()-1`).	`chain.py`
10	Display	App shows verdict, confidence, original	heatmap, action badge, tx hash, record ID. Heatmap file is saved at `outputs/heatmaps/<imageHash>.png`.
11	Later verification	Auditor uploads an image and record ID. App re-hashes the bytes, reads the record, compares `imageHash`. Equal → MATCH. Different → MISMATCH.	`chain.py`, `app.py`

Why this proves integrity: a single changed byte in the image changes the SHA-256 completely, and the contract has no function to edit or delete a record. So a later match proves the image is byte-identical to what was logged, and the existence of the record proves the decision was not changed afterwards.

What it does not prove: that the original image was a true photo of a genuine product, or that an authorised account was not itself dishonest.

B3. Technology Stack

Area	Technology	Notes
Language	Python 3.10+	
Deep learning	PyTorch, torchvision	Pretrained MobileNetV2 (`MobileNet_V2_Weights.IMAGENET1K_V1`)
Explainability	`grad-cam` (pytorch-grad-cam), `shap`, `lime`	
Metrics / plots	scikit-learn, matplotlib, pandas	
Image processing	Pillow, OpenCV	Fake generation
Blockchain dev	Node.js LTS, Hardhat	Local EVM node on `127.0.0.1:8545`, chain ID 31337, 20 funded accounts
Contract language	Solidity ^0.8.20	
Python–chain	`web3` (web3.py), `py-solc-x`	Compile with solc 0.8.20
UI	Streamlit	
Version control	Git + GitHub	
Tooling	Claude Code in terminal	Scripts generated and run locally

Ganache is **not** used (discontinued).

B4. Repository Structure

```
project/
├── data/
│   ├── raw/                        # genuine images (manual download)
│   ├── train/{genuine,counterfeit}/
│   ├── val/{genuine,counterfeit}/
│   ├── test/{genuine,counterfeit}/
│   └── fakes_manifest.csv
├── blockchain/                     # Hardhat project
│   ├── package.json
│   └── hardhat.config.js
├── contracts/MedicineAudit.sol
├── src/
│   ├── config.py
│   ├── generate_fakes.py
│   ├── dataset.py
│   ├── train.py
│   ├── explain.py
│   ├── chain.py
│   ├── hashchain.py
│   ├── audit.py
│   ├── decide.py
│   ├── deploy.py
│   ├── sweep_thresholds.py
│   ├── run_test_set.py
│   └── verify_log.py
├── tests/
│   ├── test_contract.py
│   ├── test_hashing.py
│   └── test_decide.py
├── models/best_model.pt
├── outputs/
│   ├── metrics/ gradcam/ shap/ fakes_preview/ heatmaps/ tamper_test/
│   ├── gas_and_timing.csv
│   └── hashchain_log.csv           # only if fallback used
├── contract.json                   # address + ABI (regenerated per deploy)
├── app.py
├── requirements.txt
├── README.md
└── .gitignore                      # venv/, node_modules/, data/, models/*.pt
```

B5. Data Design

Dataset

- **Genuine class:** 500–1000 images from a Kaggle pill/packaging dataset in `data/raw/`.
- **Counterfeit class:** one simulated fake per genuine image, giving a balanced 1:1 dataset (~1000–2000 images total).
- **Split:** 70/15/15 applied to the **original** images before fake generation. Example with 700 genuine: 490 / 105 / 105 genuine, plus the same counts of fakes.

`fakes_manifest.csv` schema

Column	Description
`fake_filename`	Path of generated counterfeit
`source_filename`	Genuine image it was derived from
`split`	train / val / test
`alterations`	Semicolon list, e.g. `blur_imprint;color_shift`
`strength`	low / medium / high
`localized`	true if at least one alteration affects only part of the image
`bbox`	`x1,y1,x2,y2` of the altered region (empty if global)

Leakage rules

- No base image or its derivative appears in more than one split.
- Genuine and counterfeit files use the **same final format and compression pipeline**, so file format is not a hint.

B6. Counterfeit Simulation Specification

Each fake receives a random combination of 1–3 alterations at random strength:

Alteration	Implementation	Localised?
Colour/shade shift	HSV hue ±5–20°, saturation/value ±5–25%	No (global)
Imprint blur/erase	Gaussian blur or fill/inpaint on a central or random box	Yes
Wrong/distorted text	Draw wrong or warped characters with PIL/OpenCV in a box	Yes
Shape/size change	Slight scale, aspect change, mild perspective warp	No
Noise/compression	Gaussian noise, low-quality JPEG re-encode	No

Design intent: include subtle fakes so the task is not trivial (a model scoring ~100% undermines credibility). The report must state counterfeits are simulated and why.

B7. Model Specification

Parameter	Value
Architecture	MobileNetV2 (ResNet18 acceptable alternative) pretrained on ImageNet
Head	Final classifier layer replaced with `Linear(in_features, 2)`
Input size	160×160 (224×224 on GPU)
Normalisation	ImageNet mean `[0.485, 0.456, 0.406]`, std `[0.229, 0.224, 0.225]`
Augmentation (train)	Horizontal flip, rotation ±10–15°. No colour jitter if colour is a counterfeit cue
Loss	Cross-entropy
Optimiser	Adam
Learning rate	1e-3 (head), 1e-4 if fine-tuning last block
Batch size	32
Epochs	6–10 (CPU) / 10–15 (GPU)
Training stages	Stage 1: backbone frozen, head only. Stage 2 (optional): unfreeze last block
Checkpoint rule	Best validation **counterfeit recall** (F1 tie-break)
Seeds	`torch`, `numpy`, `random` fixed in `config.py`

Class-index detail (important): `ImageFolder` assigns indices alphabetically, so `counterfeit = 0` and `genuine = 1`. P(counterfeit) is therefore `softmax(logits)[:, 0]`. Save `class_to_idx` in the checkpoint and use it everywhere, so the probability is never read from the wrong class.

Evaluation outputs (test set): accuracy, per-class precision/recall/F1, counterfeit recall, ROC-AUC, confusion matrix image, training curves, `metrics.json`.

B8. Explainability Specification

Grad-CAM (primary)

- Library: `pytorch_grad_cam.GradCAM(model=model, target_layers=[model.features[-1]])`.
- Target: counterfeit class (or predicted class).
- Output: float CAM map (0–1) and an overlay via `show_cam_on_image`; overlay encoded to PNG bytes.

Explanation validity check

- For counterfeit test images with `localized = true`, build a mask from `bbox`.
- Metrics: (a) share of total CAM mass inside the mask, (b) IoU between top-20% CAM pixels and the mask.
- Report mean and distribution; show good and bad cases.
- Do **not** apply this metric to purely global alterations (the mask would be the whole image, making the score meaningless).
- Control: on genuine images, report that CAM is diffuse or not tied to a fake region.

SHAP / LIME (secondary)

- Run on 10–20 test images (LIME image explainer is the CPU-friendly option if SHAP is slow).
- Compare to Grad-CAM on speed per image, overlap score, readability, and stability (same image, re-run).

Output: 6–8 curated figures (TP, TN, FP, FN) in `outputs/gradcam/` and `outputs/shap/`.

B9. Decision Logic Specification

```
p = P(counterfeit)
if p >= T_HIGH (0.85):  FLAG     verdict=1
elif p <= T_LOW (0.15): APPROVE  verdict=0
else:                   ESCALATE verdict=2
```

Aspect	Specification
Thresholds	Default 0.85 / 0.15 in `config.py`
Tuning	`sweep_thresholds.py` evaluates many (T_LOW, T_HIGH) pairs on **validation**; outputs automation rate, wrong auto-approve count, wrong auto-flag count
Selection	Choose thresholds that minimise missed fakes among APPROVE decisions while keeping escalation acceptable; confirm once on test
Rationale	Missing a fake is costlier than a false alarm, so the APPROVE bound is the stricter one
Output	`(action, verdict, probability, heatmap_bytes, receipt)`

B10. Hashing and Audit Data Specification

Item	Specification
Hash function	SHA-256 via `hashlib.sha256(data).digest()` → 32 bytes → Solidity `bytes32`
Image hash input	**Original uploaded bytes**, never a re-encoded or re-saved image (re-encoding would create false mismatches)
Heatmap hash input	Exact bytes of the saved heatmap PNG file
Heatmap storage	`outputs/heatmaps/<imageHashHex>.png`, so it can be re-hashed and compared later
Confidence encoding	`int(round(p * 10000))`, fits in `uint16` (9250 = 92.50%)
Verdict encoding	0 genuine, 1 counterfeit, 2 escalate
Timestamp	`block.timestamp` set by the contract (not trusted from the client)

B11. Smart Contract Specification (`contracts/MedicineAudit.sol`)

Data

```
struct Record { bytes32 imageHash; bytes32 heatmapHash; uint8 verdict;
                uint16 confidence; uint256 timestamp; address submitter; }
address public owner;
mapping(address => bool) public authorized;
Record[] private records;
event DecisionLogged(uint256 indexed id, bytes32 imageHash, uint8 verdict);
```

Functions

Function	Access	Behaviour
`constructor()`	deploy	Sets `owner = msg.sender`, authorises owner
`authorize(address)`	owner only	Grants logging rights
`revoke(address)` (recommended)	owner only	Removes logging rights
`logDecision(imgH, heatH, verdict, conf)`	authorised	Validates `verdict <= 2`, appends Record, emits event, returns ID
`getRecord(uint256)`	public view	Returns record; reverts if ID out of range
`totalRecords()`	public view	Returns count

Properties

- **Append-only:** no update or delete function exists, which provides tamper evidence.
- **Access control:** `onlyAuthorized` modifier on `logDecision`.
- **Gas:** each record writes about five storage slots, so expect roughly 10^5 gas per record. Measure and report the actual figure from receipts.
- **Events:** allow efficient indexing and later retrieval by ID.

Contract tests (`tests/test_contract.py` or Hardhat tests)

1. Owner can log, ID starts at 0, `totalRecords` increments.
2. Unauthorised account reverts with "Not authorized".
3. `authorize` by non-owner reverts.

4. `getRecord` returns exactly what was stored.
5. Invalid verdict (> 2) reverts.
6. Out-of-range `getRecord` reverts.
7. After `revoke`, the account can no longer log.

B12. Python–Blockchain Integration Specification (`src/chain.py`)

Function	Behaviour
`connect()`	`Web3(Web3.HTTPProvider(url))`; raises `AuditUnavailable` if `is_connected()` is false
`compile_contract()`	`install_solc("0.8.20")`, `compile_files(...)`, returns ABI and bytecode
`deploy()`	Deploys from `w3.eth.accounts[0]`, waits for receipt, writes `contract.json` (address, ABI, chain ID, deploy block)
`load_contract()`	Reads `contract.json`; checks `w3.eth.get_code(address)` is non-empty, else raises `StaleContract` (node was restarted)
`log_decision(image_bytes, heatmap_bytes, verdict, conf)`	Hashes, converts confidence, sends transaction, waits for receipt; returns `{tx_hash, record_id, gas_used, block_number, seconds}`
`verify(image_bytes, record_id)`	Re-hashes, reads on-chain record, returns `{match: bool, stored_hash, computed_hash, record}`
`get_all_records()`	Loops `0..totalRecords-1`, returns list of dicts for the ledger

Custom exceptions: `AuditUnavailable`, `StaleContract`, `NotAuthorized`, `InvalidInput`.

`src/deploy.py` is the single command to run after every node restart: check node → compile → deploy → write `contract.json` → print address.

B13. Fallback Backend Specification (`src/hashchain.py`)

Used only if the chain is not working after the 2-day rule.

- CSV columns: `timestamp, filename, image_hash, heatmap_hash, verdict, confidence, prev_hash, row_hash`.

- `row_hash = SHA256(prev_hash + timestamp + filename + image_hash + heatmap_hash + verdict + confidence)`; genesis `prev_hash = "0"*64`.
- Append-only writes; `verify_log.py` recomputes the chain and reports the first broken row.
- Same public interface as `chain.py`: `log_decision`, `verify`, `get_all_records`.
- `src/audit.py` imports the backend named by `config.AUDIT_BACKEND`.

B14. Application Specification (`app.py`)

Tab	Inputs	Outputs
Check medicine	Image upload	Verdict, confidence, original
Blockchain ledger	Refresh button	Table of records, CSV download
Verify record	Image upload + record ID	MATCH / MISMATCH, stored vs computed hash, optional heatmap-file check
About	None	Metrics, confusion matrix, thresholds, limitations (simulated data)

Sidebar: chain status, contract address, active account, thresholds, backend in use. Implementation notes: `st.cache_resource` for model and web3 connection; `st.tabs`; `st.spinner` during processing; all exceptions caught and shown with `st.error`.

B15. Configuration (`src/config.py`)

```
SEED=42; IMG_SIZE=160; BATCH_SIZE=32; EPOCHS=8; LR=1e-3
T_HIGH=0.85; T_LOW=0.15
SPLIT=(0.70, 0.15, 0.15)
AUDIT_BACKEND="chain"            # or "hashchain"
RPC_URL="http://127.0.0.1:8545"
CONTRACT_FILE="contract.json"
MODEL_PATH="models/best_model.pt"
```

B16. Error Handling Matrix

Condition	Detection	Behaviour
Not an image / corrupt	PIL raises	`st.error("Invalid image")`, nothing logged
Node not running	`is_connected()` false	Show verdict and heatmap, warn "Not recorded on blockchain: start Hardhat node"
Stale contract after restart	`get_code` empty	Message "Run `python src/deploy.py`"
Unauthorised account	Transaction revert	Clear "not authorised" message
Out-of-range record ID	Contract revert	"Record does not exist"
Model file missing	File check	Message to run training
Transaction timeout	Receipt wait timeout	Message with retry option

B17. Security and Privacy Considerations

- Only hashes on-chain; images stay local.
- Hardhat private keys are public test keys. Never reuse them or commit real keys; load any real-key concept from environment variables in future work.
- Access control limits who can write, but a compromised authorised account could log false records (state in limitations).
- Verification requires the exact original bytes; any re-save fails by design.
- Local chain gives no real decentralisation; on a public chain you would also consider gas cost, finality time, public visibility of hashes, and key management.
- Model can be fooled by adversarial or out-of-distribution images; escalation partly mitigates this.

B18. Performance Targets

Stage	Target (CPU laptop)
Preprocess + inference	< 0.5 s
Grad-CAM	< 1 s
Hashing	< 0.05 s
Chain transaction (Hardhat automine)	< 1–2 s
Total per decision	⩽ ~5 s
Training (frozen backbone, 160px, ~1500 images)	~10–30 min

These are targets; record actual values in `gas_and_timing.csv`.

B19. Testing Strategy

Level	Tests
Unit	Hash function consistency, confidence encoding, threshold mapping, class-index handling
Data	No leakage across splits, class balance, manifest completeness
Model	Tiny-batch overfit, baseline comparison, shortcut checks (file size, dimensions, JPEG quality)
Explainability	Overlap metric on localised fakes, genuine-image control
Contract	The seven tests in B11
Integration	log → verify MATCH; modify image → MISMATCH; unauthorised → revert; node down → graceful message; node restart → stale-contract message
System	Full viva script end to end
Performance	Gas and time per decision over the whole test set

B20. Deployment and Run Procedure

```
# Terminal 1: local chain
cd blockchain && npx hardhat node

# Terminal 2: project root (venv active)
python src/deploy.py          # after every node restart
streamlit run app.py
```

B21. Reproducibility

Fixed seeds, pinned `requirements.txt`, saved `class_to_idx`, saved split lists, saved manifest, saved metrics JSON, and a README with exact commands.

PART C: THE 12-PHASE IMPLEMENTATION PLAN

Each phase lists: objective, owner, duration, what happens, how it is done, files, Claude Code prompt, deliverables, exit criteria, and pitfalls.

Phase 1: Planning, Scope Lock, and Repository Setup

Owner: Both | **Days:** 1

Objective: Agree on what is being built, divide the work, and create the project skeleton.

What happens

- The scope (Option C) is confirmed with the guide.
- Roles are assigned: Person A = data, model, explainability; Person B = contract, web3, app.
- A fallback rule is written down: if blockchain integration is not working after 2 days of effort, switch to the hash-chain backend.

How

1. Meet your guide, present the architecture diagram (B1), and get approval. Ask for the deadline.
2. Create the GitHub repo; add collaborator; create branches `data-model` and `chain-app`.
3. Create the folder structure (B4) with empty placeholders and a `.gitignore`.
4. Create `README.md` with project title and the run order (fill in later).
5. Start the **logbook** (weekly entries from now on).

Claude Code prompt: "Create the project skeleton exactly as in the structure, with `.gitignore`, empty `__init__` files where needed, and a starter `README.md`."

Deliverables: Repo with skeleton, written role split, guide approval. **Exit criteria:** Both members can clone, push, and see the structure. **Pitfalls:** Starting code before agreeing the fallback rule; committing `data/` or `node_modules/` (add to `.gitignore`).

Phase 2: Environment Setup (Python, Node, Hardhat)

Owner: Both | **Days:** 1–2

Objective: Every tool needed runs on both laptops.

What happens

- Python virtual environment with all libraries is created.
- Node.js and Hardhat are installed, and a local blockchain is started and reached from Python.
- Hardware is checked to choose training settings.

How

1. `nvidia-smi`: GPU present? Choose settings from the hardware table (B7). No GPU → install the CPU build of torch.
2. Create the venv and install: `pip install torch torchvision grad-cam shap lime streamlit scikit-learn matplotlib pillow opencv-python pandas web3 py-solc-x` then `pip freeze > requirements.txt`.
3. Install Node LTS. In `blockchain/`: `npm init -y`, `npm install --save-dev hardhat`, `npx hardhat init` (empty config).
4. Start `npx hardhat node` and keep it open.
5. Smoke test in Python: `Web3(Web3.HTTPProvider("http://127.0.0.1:8545")).is_connected()` is `True`, and `w3.eth.accounts` has 20 entries.
6. Pre-download the compiler once: `from solcx import install_solc; install_solc("0.8.20")`, so a slow download does not block you later.
7. Write `src/config.py` (B15).

Claude Code prompt: "Write *`src/config.py`* and a *`src/smoke_test.py`* that checks imports, prints torch device, and checks the Hardhat node connection and account count."

Deliverables: `requirements.txt`, working node, smoke test passing. **Exit criteria:** `smoke_test.py` prints all green on both machines. **Pitfalls:** Antivirus or firewall blocking port 8545; forgetting to activate the venv; Windows path issues (use forward slashes or `pathlib`).

Phase 3: Dataset Acquisition and Cleaning

Owner: A | **Days:** 2–3

Objective: A clean set of 500–1000 genuine medicine images in `data/raw/`.

What happens

- A public Kaggle dataset is downloaded manually (Claude Code cannot log in to Kaggle).
- Images are inspected, filtered, and prepared as the "genuine" class.

How

1. In your browser, search Kaggle for "pill images", "medicine packaging", "tablet imprint"; pick one with clear images and a usable licence. Note name, URL, licence, and image count for the report.
2. Unzip into `data/raw/`.
3. Run a cleaning script: remove unreadable files, images below a minimum size (e.g., < 64 px), exact duplicates (hash-based), and non-image files.
4. If there are more than ~1000, randomly select 1000 (seeded) to keep CPU training manageable.
5. Convert all to a single format (e.g., RGB JPEG at a fixed quality or PNG) using one consistent pipeline.
6. Produce a summary: count, size distribution, a sample montage image.

Claude Code prompt: "Look at `data/raw`. Write `src/clean_dataset.py` that removes corrupt, tiny, duplicate, and non-image files, standardises format, caps at 1000 images with a fixed seed, and prints a summary and saves a sample montage."

Deliverables: Clean `data/raw/`, dataset description paragraph (source, licence, count). **Exit criteria:** 500–1000 valid images; summary saved. **Pitfalls:** Images that are mostly background; near-duplicates from the same photo session (consider noting this as a leakage limitation); mixed formats later acting as shortcuts.

Phase 4: Counterfeit Simulation and Dataset Splits

Owner: A | **Days:** 3–5

Objective: A balanced, leak-free dataset in `data/train|val|test/{genuine,counterfeit}` plus a manifest.

What happens

- Genuine images are split **first** (70/15/15).

- For each genuine image in each split, a counterfeit is synthesised by random alterations (B6).
- A manifest records exactly what was changed and where, which later validates Grad-CAM.

How

1. Seed the RNG; shuffle; split originals 70/15/15; copy to `data/<split>/genuine/`.
2. For each, choose 1–3 alterations and a strength; apply with PIL/OpenCV:
   - colour shift in HSV space
   - blur or fill a box over the imprint region
   - draw wrong text in a box
   - slight scale, aspect, or perspective change
   - noise, then JPEG re-encode
3. For localised alterations, store the `bbox`.
4. Save with the **same final format and quality settings as the genuine images** (for example, re-save genuine images through the same pipeline).
5. Write `fakes_manifest.csv`.
6. Generate a preview grid (genuine vs fake pairs) for `outputs/fakes_preview/`.
7. Run assertions: counts match, classes balanced, **no source filename appears in two splits**.

Claude Code prompt: "Write `src/generate_fakes.py` that splits `data/raw` 70/15/15 first, then creates one counterfeit per genuine image with random combinations of colour shift, imprint blur/erase, wrong text, shape distortion, and noise/compression at varied strengths. Save a manifest CSV with alterations and bounding boxes, a preview grid, and assert no split leakage."

Deliverables: Populated split folders, manifest, preview grid. **Exit criteria:** Assertions pass; preview shows believable, varied fakes (some subtle, some obvious). **Pitfalls:** Alterations too obvious (model scores 100%); different compression between classes (shortcut); data leakage via duplicates.

Phase 5: CNN Training and Evaluation

Owner: A | **Days:** 5–9

Objective: A trained MobileNetV2 that detects counterfeits with high recall, with full metrics.

What happens

- Images are loaded and augmented; a pretrained network is adapted to 2 classes; trained; best checkpoint kept by counterfeit recall; evaluated on the untouched test set.

How

1. `dataset.py`: `ImageFolder` datasets with train/eval transforms (B7), `DataLoader`s (batch 32, `num_workers=0` on Windows).
2. `train.py`:
   - Load `mobilenet_v2(weights=MobileNet_V2_Weights.IMAGENET1K_V1)`.
   - Replace `classifier[1]` with `Linear(1280, 2)`.
   - Freeze `features`; train the head with Adam, lr=1e-3.
   - Each epoch: compute train and val loss/accuracy and val counterfeit recall; save the best checkpoint including `class_to_idx`.
   - Optional stage 2: unfreeze the last block, lr=1e-4, 2–3 epochs.
3. Evaluate on test: accuracy, precision/recall/F1 per class, ROC-AUC, confusion matrix; save `metrics.json`, `confusion_matrix.png`, `training_curves.png`.
4. Sanity checks:
   - Overfit 16 images quickly to prove the loop works.
   - Train a trivial baseline (e.g., colour-histogram + logistic regression) and compare.
   - Check shortcuts: does file size or image dimension alone predict the class?
5. Write down why recall on counterfeit is prioritised.

Claude Code prompt: "Write *`src/dataset.py`* and *`src/train.py`*: MobileNetV2 transfer learning at 160x160, CPU-friendly, frozen backbone, light augmentation, 8 epochs, save best model by validation counterfeit recall with class_to_idx, then print and save accuracy, precision, recall, F1, ROC-AUC, confusion matrix and training curves to outputs/metrics/."

Deliverables: `models/best_model.pt`, `outputs/metrics/*`. **Exit criteria:** Test counterfeit recall ⩾ 0.90 (or documented reason), baseline comparison done, no obvious shortcut. **Pitfalls:** Reading probability from the wrong class index; evaluating on validation data and calling it test; forgetting `model.eval()` at inference.

Phase 6: Explainability (Grad-CAM, SHAP/LIME, Validation)

Owner: A | **Days:** 9–12

Objective: Show why the model decides, and provide evidence the explanations are meaningful.

What happens

- Grad-CAM highlights regions that drove the counterfeit score.
- Highlighted regions are compared to the known altered regions from the manifest.
- A second method (SHAP or LIME) is compared against Grad-CAM.

How

1. `explain.py`: implement `gradcam(image) → (overlay_rgb, cam_array)` using `GradCAM(model, target_layers=[model.features[-1]])`.
2. Add `gradcam_png_bytes(image)` that returns the overlay encoded as PNG bytes (used by the pipeline).
3. Overlap validation: for each localised fake in the test set, compute CAM mass inside `bbox` and IoU of top-20% CAM with `bbox`. Record mean and a histogram. Compute a control on genuine images.
4. SHAP/LIME: run on 10–20 images; produce triptych figures (original | Grad-CAM | SHAP/LIME).
5. Comparison table: time per image, mean overlap, qualitative readability, stability on repeated runs.
6. Curate 6–8 figures covering true positive, true negative, false positive, false negative; save to `outputs/gradcam/` and `outputs/shap/`.

Claude Code prompt: "Write *`src/explain.py`*: Grad-CAM overlays for sample test images, a function returning the overlay as PNG bytes, an overlap score (CAM mass and top-20% IoU) against the manifest bboxes for localised fakes, plus a LIME or SHAP comparison on 10 images saved as side-by-side figures and a comparison table CSV."

Deliverables: Heatmap function, overlap stats, 6–8 figures, comparison table. **Exit criteria:** Overlap statistics computed and interpreted honestly; figures saved. **Pitfalls:** Wrong target layer (heatmap looks blocky or meaningless); applying overlap to global alterations; over-claiming that heatmaps "prove" correctness.

Phase 7: Smart Contract Development and Testing

Owner: B | **Days:** 9–13 (parallel with Phases 5–6)

Objective: A tested `MedicineAudit` contract with append-only storage and access control.

What happens

- A Solidity contract is written that stores decision records and only lets authorised accounts add them.
- Tests confirm both the normal behaviour and the failure cases.

How

1. Write `contracts/MedicineAudit.sol` per B11 (struct, owner, `authorized` mapping, `logDecision`, `getRecord`, `totalRecords`, plus recommended `revoke`, verdict validation, and bounds check).
2. Compile with py-solc-x (`solc 0.8.20`) to check syntax.
3. Write `tests/test_contract.py` using web3.py against the running Hardhat node (or Hardhat's own JS tests), covering the seven tests in B11.
4. Manually exercise: log from account 0, try from account 1 (expect revert), authorise account 1, retry (success), revoke, retry (revert).
5. Record the gas used per `logDecision` call.

Claude Code prompt: "Write *`contracts/MedicineAudit.sol`* with an owner, an authorised-accounts mapping, an append-only Record array, *`logDecision`* with verdict validation, *`getRecord`* with a bounds check, *`totalRecords`*, *`authorize`*, *`revoke`*, and a *`DecisionLogged`* event. Then write *`tests/test_contract.py`* covering authorised log, unauthorised revert, non-owner authorise revert, stored values, invalid verdict, out-of-range read, and revoke."

Deliverables: Contract file, passing tests, gas figure. **Exit criteria:** All contract tests pass. **Pitfalls:** `Record memory` return requires the right ABI decoding in Python (returned as a tuple); forgetting that restarting the node wipes the deployment.

Phase 8: Python–Blockchain Integration and Fallback Backend

Owner: B | **Days:** 13–16

Objective: Python can deploy, write to, read from, and verify against the contract through a clean two-function interface, with a drop-in fallback.

What happens

- `chain.py` wraps all web3 operations.
- `audit.py` exposes `log_decision` and `verify` and routes to the chosen backend.
- `hashchain.py` provides the same functions using a hash-chained CSV, as insurance.

How

1. `chain.py`: implement the functions in B12 (`connect`, `compile_contract`, `deploy`, `load_contract`, `log_decision`, `verify`, `get_all_records`) and the custom exceptions.
2. Confidence conversion: `int(round(p*10000))`. Hashes: `hashlib.sha256(data).digest()`.
3. In `log_decision`, build and send the transaction from the configured account, wait for the receipt, read `gasUsed`, `blockNumber`, and the new record ID (from `DecisionLogged` event or `totalRecords()-1`); measure elapsed seconds.

4. `deploy.py`: one command that checks the node, compiles, deploys, writes `contract.json`, prints the address.
5. Stale-contract detection: if `get_code(address)` is empty, raise `StaleContract` with the message "Run python src/deploy.py".
6. `hashchain.py` and `audit.py` per B13.
7. Integration test script: log a sample, verify (MATCH), flip a byte in the image, verify (MISMATCH), try unauthorised account (revert), stop node (graceful error).
8. **Day-15 checkpoint:** if this is not working after 2 days of effort, set `AUDIT_BACKEND="hashchain"` and move on.

Claude Code prompt: "Write *`src/chain.py`* and *`src/deploy.py`* using web3.py and py-solc-x: compile *`contracts/MedicineAudit.sol`*, deploy from the first Hardhat account, save *`contract.json`*, and implement *`log_decision(image_bytes, heatmap_bytes, verdict, conf)`* returning tx hash, record id, gas used, block number and seconds; *`verify(image_bytes, record_id)`*; and *`get_all_records()`*. Add custom exceptions and stale-contract detection. Then write *`src/hashchain.py`* with the same interface and *`src/audit.py`* that selects the backend from config."

Deliverables: `chain.py`, `deploy.py`, `hashchain.py`, `audit.py`, integration test results. **Exit criteria:** Log → verify MATCH; altered image → MISMATCH; backend swap works by changing one config value. **Pitfalls:** Hashing a re-encoded image instead of the original bytes; nonce errors from sending transactions too fast; forgetting to redeploy after restarting the node.

Phase 9: Decision Pipeline and Threshold Tuning

Owner: Both | **Days:** 16–18

Objective: One function that goes from image bytes to decision, explanation, and audit record, with justified thresholds.

What happens

- The model, Grad-CAM, decision rule, and audit backend are joined into `decide()`.
- Thresholds are tuned on validation data, not guessed.

How

1. `decide.py`: implement the flow in B2 (validate → preprocess → predict → heatmap → action → save heatmap file → audit log → return result). Catch `AuditUnavailable`, `StaleContract`, `NotAuthorized` and return a result with an `error` field instead of crashing.

2. Load the model once (module-level cache) with `class_to_idx` from the checkpoint.
3. Time each stage and return the timings.
4. `sweep_thresholds.py`: for a grid of (T_LOW, T_HIGH), compute on validation: % APPROVE/FLAG/ESCALATE, number of fakes auto-approved (worst error), genuine items auto-flagged. Plot automation rate vs. error counts; choose thresholds and justify.
5. `run_test_set.py`: run `decide()` on every test image (with logging) and save `outputs/gas_and_timing.csv` plus the action counts.
6. Unit tests for threshold mapping at boundaries (0.15, 0.85) and invalid input.

Claude Code prompt: "Write *`src/decide.py`* combining model prediction, Grad-CAM, threshold logic (approve/flag/escalate), saving the heatmap file, logging via *`audit.log_decision`*, per-stage timing, and graceful handling of chain errors. Then write *`src/sweep_thresholds.py`* and *`src/run_test_set.py`* as described."

Deliverables: `decide.py`, threshold plot, `gas_and_timing.csv`, action counts. **Exit criteria:** Full test set runs through the pipeline; thresholds justified; boundary tests pass. **Pitfalls:** Choosing thresholds on the test set (leakage); heatmap hashed before saving (hash must be of the saved file bytes).

Phase 10: Streamlit Web Application

Owner: B (A reviews) | **Days:** 18–21

Objective: A demonstrable interface for all user flows.

What happens

- A four-tab app exposes checking, ledger browsing, verification, and project information.

How

1. Layout: `st.tabs(["Check medicine","Blockchain ledger","Verify record","About"])`; sidebar with chain status, contract address, active account, thresholds, backend.
2. **Check tab:** `st.file_uploader`; on upload call `decide(bytes)` inside `st.spinner`; show original and heatmap with `st.columns`; coloured badge (`st.success` / `st.error` / `st.warning`); metrics for confidence; tx hash and record ID; timing; plain-language sentence derived from the action.
3. **Ledger tab:** call `get_all_records()`; show dataframe (truncated hashes, readable verdict labels, formatted time); refresh button; CSV download.
4. **Verify tab:** upload image + `number_input` for record ID; call `verify`; show green MATCH or red MISMATCH and both hashes; optional heatmap-file check.

5. **About tab:** metrics JSON, confusion matrix image, thresholds, limitations list including the simulated-data disclaimer.
6. Wrap every call in try/except and show friendly messages (B16). Use `st.cache_resource` for model and web3.

Claude Code prompt: "Write `app.py`: a Streamlit app with tabs Check medicine, Blockchain ledger, Verify record, and About, plus a sidebar showing chain status, contract address, and thresholds. Use `decide()` and `audit` functions, show original and Grad-CAM side by side, a colour-coded action, tx hash and record id, handle all errors gracefully."

Deliverables: Working `app.py`, screenshots of each tab. **Exit criteria:** Full demo flow (upload → record → ledger → verify MATCH/MISMATCH) runs without exceptions; node-down case shows a friendly message. **Pitfalls:** Streamlit reruns re-executing the transaction (guard with `st.session_state` or a button); caching a stale web3 connection after node restart.

Phase 11: Testing, Evaluation, and Robustness

Owner: Both | **Days:** 21–25

Objective: Produce all evidence needed for the Results section and confirm everything works.

What happens

- The system is tested at unit, integration, and system level; results are collected into `outputs/`.

How

Test	Procedure	Output
Model metrics	Final test-set run	Confusion matrix, metrics table
Explanation quality	Overlap stats, figures	Table + 6–8 figures
Method comparison	Grad-CAM vs SHAP/LIME	Comparison table
Tamper test	Log a record, edit the image (one pixel or re-save), run `verify`	Screenshots: MATCH on original, MISMATCH on edited
Access control	Unauthorised account tries `logDecision`	Screenshot of revert
Decision statistics	Run test set	Counts and % approved/flagged/escalated, wrong auto-decisions
Cost and time	From receipts and timers	`gas_and_timing.csv` summary (mean, max)
Failure handling	Stop node; restart node without redeploying; upload a text file renamed `.jpg`	Screenshots of friendly messages
Robustness	Hold out one alteration type from training and test on it; try phone-style photos if available	Short table showing generalisation limits
Reproducibility	Fresh clone, follow README	README verified

Claude Code prompt: "Write `src/tamper_demo.py` that logs a sample image, verifies it (MATCH), modifies one pixel, verifies again (MISMATCH), saves both results to `outputs/tamper_test/`, and then summarise all files in `outputs/` into a results table."

Deliverables: Completed `outputs/`, results table, screenshots. **Exit criteria:** Every item in the table has an artefact; limitations are clear. **Pitfalls:** Reporting only the good results; skipping the robustness test that exposes limitations (a short honest discussion scores better than inflated claims).

Phase 12: Report, Logbook, Final Packaging, and Viva

Owner: Both | **Days:** 24–28

Objective: A complete report, clean repository, and a rehearsed demo.

What happens

- All results are written up, the repository is finalised, and the viva demo is rehearsed.

How

1. **Report outline**
    1. Introduction (problem, objectives)
    2. Literature Review (existing document)
    3. System Design (architecture diagram, data flow from B2, PRD summary)
    4. Methodology (dataset and simulation, model, explainability, decision logic, hashing, smart contract)
    5. Implementation (tools, code structure, app screenshots)
    6. Results (metrics, explanations, decision stats, tamper test, gas/time)
    7. Limitations and Discussion
    8. Conclusion and Future Work
2. **Limitations to state honestly:** simulated counterfeits; small dataset; local test network; chain proves record immutability but not input authenticity; compromised authorised account risk.
3. **Future work:** public testnet (e.g., Sepolia), real counterfeit data via partners, supply-chain batch registration, IPFS for image storage, spectroscopy fusion, confidence calibration.
4. Finalise README (setup, run order, screenshots), tag a release, and back up the repo.
5. Update the logbook; get guide sign-off.
6. **Viva demo script (≈5 min):**
    1. Show architecture slide.
    2. Upload a genuine image → APPROVE + tx hash.
    3. Upload an obvious fake → FLAG + heatmap on the altered spot.
    4. Upload a borderline image → ESCALATE.
    5. Show ledger tab.
    6. Verify original → MATCH; verify an edited copy → MISMATCH.
    7. Show unauthorised-account revert screenshot and metrics slide.

7. Rehearse answers (Appendix). Prepare a **backup**: screen recording and screenshots in case the node or laptop misbehaves.

Claude Code prompt: "Help me write the Results section using the files in `outputs/`, then draft the Limitations and Future Work sections."

Deliverables: Final report, repo with README, demo recording, viva notes. **Exit criteria:** Two full rehearsals complete with no errors. **Pitfalls:** Leaving report writing to the last two days; not testing the demo on the presentation machine; forgetting to start the node and deploy before the viva.

PART D: TIMELINE, RISKS, DEFINITION OF DONE

D1. Timeline

Phase	Name	Owner	Days
1	Planning and repo	Both	1
2	Environment setup	Both	1–2
3	Dataset acquisition	A	2–3
4	Fake simulation and splits	A	3–5
5	Model training and evaluation	A	5–9
6	Explainability	A	9–12
7	Smart contract	B	9–13
8	Python–chain integration + fallback	B	13–16
9	Decision pipeline and thresholds	Both	16–18
10	Streamlit app	B	18–21
11	Testing and evaluation	Both	21–25
12	Report, logbook, viva	Both	24–28

Person B starts Phase 7 on Day 9 in parallel with Person A's Phases 5–6; before Day 9, Person B helps with data work and sets up Hardhat. **Checkpoint:** Day 15 blockchain fallback decision.

D2. Risks and Mitigations

Risk	Impact	Mitigation
Blockchain tooling consumes time	Schedule slip	2-day rule; hash-chain fallback behind the same interface
Hardhat node resets	Stale contract, errors	One-command `deploy.py`; stale detection in app
Solidity compiler download fails	Cannot compile	Pre-install in Phase 2; or compile via Hardhat and read artifact JSON
No GPU	Slow training	MobileNetV2, 160px, frozen backbone, capped dataset
Fakes too easy	Unconvincing metrics	Varied strength, subtle fakes, honest discussion
Shortcut learning	Misleading accuracy	Same save pipeline for both classes; shortcut checks
Data leakage	Inflated metrics	Split originals first; assertions
Wrong class index	Inverted probabilities	Store and use `class_to_idx`; unit test
Hash mismatch from re-encoding	False tamper alarms	Hash original uploaded bytes only
SHAP slow on CPU	Delay	LIME or fewer images
Grad-CAM poorly aligned	Weak explainability claim	Report honestly; try other layers
Demo failure in viva	Lost marks	Backup recording; rehearse; start-up checklist
Windows DataLoader errors	Crashes	`num_workers=0`; `if __name__ == "__main__":`

D3. Definition of Done

- [ ] Repository with skeleton, README, `requirements.txt`
- [ ] Clean dataset, leak-free 70/15/15 splits, manifest
- [ ] Trained model with metrics, confusion matrix, baseline comparison
- [ ] Grad-CAM with overlap validation; SHAP/LIME comparison; 6–8 figures

- [ ] Smart contract with access control and passing tests
- [ ] `chain.py`, `deploy.py`, working `log_decision()` / `verify()`
- [ ] Hash-chain fallback behind the same interface
- [ ] `decide.py` with justified thresholds and graceful error handling
- [ ] Streamlit app with all four tabs
- [ ] Tamper test and access-control screenshots
- [ ] Gas and timing table; approve/flag/escalate statistics
- [ ] Report written with limitations stated honestly
- [ ] Logbook entries complete
- [ ] Viva demo rehearsed with a backup recording

APPENDIX

Viva Q&A Preparation

Question	Key points for the answer
Why simulate counterfeits?	No public labelled counterfeit dataset exists; simulation tests the pipeline, not real-world accuracy; stated as a limitation
Why transfer learning?	Small dataset; pretrained features generalise; faster on CPU
Why prioritise counterfeit recall?	A missed fake can harm patients; false alarms are cheaper
What does Grad-CAM show, and why also SHAP?	Grad-CAM shows regions that increased the class score; SHAP/LIME give an independent view; agreement increases trust; comparison on speed and overlap
How did you validate the explanations?	Overlap with known altered regions from the manifest; control on genuine images
How did you choose thresholds?	Sweep on validation; strict APPROVE bound; confirmed on test
Why store hashes, not images?	Chain storage is expensive and public; a hash is a compact tamper-proof fingerprint
Why a local network? What changes on a public chain?	Free, fast, no risk; public chain adds gas cost, slower finality, public visibility, key management
How is it tamper-evident, and what can it not protect against?	Append-only contract plus hash comparison detects later changes; cannot prove the original input was genuine or stop a compromised authorised account
What happens if the blockchain is down?	App still shows verdict and heatmap, warns it was not recorded; fallback backend exists
Why hash the original bytes?	Re-encoding changes bytes and would cause false mismatches
What would you do next?	Real counterfeit data, public testnet, IPFS, batch registration, calibration

Glossary

Term	Meaning
CNN	Convolutional neural network for image classification
Transfer learning	Reusing a network pretrained on ImageNet and adapting its last layer
Grad-CAM	Heatmap method using gradients flowing into the last conv layer
SHAP / LIME	Model-agnostic explanation methods attributing predictions to image regions
Recall (counterfeit)	Share of real counterfeits that the model catches
Escalate	Send to a human reviewer when the model is uncertain
SHA-256	Cryptographic hash producing a 32-byte fingerprint
Smart contract	Program stored and executed on a blockchain
Hardhat	Local Ethereum development node and tooling
web3.py	Python library to talk to an Ethereum node
Gas	Unit of computation cost for blockchain transactions
Tamper-evident	Changes can be detected, though not necessarily prevented
Hash chain	Each log row includes the hash of the previous row, so edits break the chain